Etude de cas KANBAN Jayde

Étape 1 – Analyse de l'existant

Question 1

npm install fonctionne. npm run dev ne démarre que si un fichier .env contient MONGODB_URI et JWT_SECRET. En revanche, npm test échouait : aucune variable d'environnement n'était fournie aux tests, donc l'application s'arrêtait au chargement. npm run build échouait aussi (erreur TypeScript TS6059, car les tests étaient inclus dans la compilation).

```
❌ Invalid environment variables:
 - MONGODB_URI: Invalid input: expected string, received undefined
 - JWT_SECRET: Invalid input: expected string, received undefined
Error: process.exit unexpectedly called with "1"
```

Question 2

Express 5 pour le serveur, Mongoose 8 pour MongoDB, jsonwebtoken et bcryptjs pour l'authentification, Zod pour la validation, ainsi que cors, dotenv et pino. Helmet et express-rate-limit étaient installés mais jamais utilisés. Côté tests : Vitest, Supertest et MongoDB Memory Server.



```
"dependencies": {
  "bcryptjs": "^3.0.2",
  "cors": "^2.8.5",
  "dotenv": "^17.2.3",
  "express": "^5.1.0",
  "express-rate-limit": "^8.2.1",
  "helmet": "^8.1.0",
  "jsonwebtoken": "^9.0.2",
  "mongoose": "^8.19.2",
  "pino": "^10.1.0",
  "pino-pretty": "^13.1.2",
  "zod": "^4.1.12"
},
"devDependencies": { "vitest": "^5.0.2", "supertest": "^7.1.4", "mongodb-memory-server": "^10.2.3" }
```

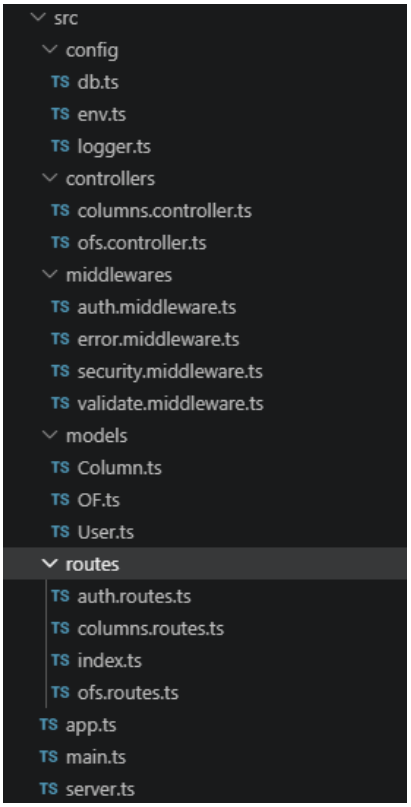
Question 3

MONGODB_URI et JWT_SECRET (32 caractères minimum) sont obligatoires. NODE_ENV, PORT (3000), JWT_EXPIRES_IN (7 jours) et CORS_ORIGIN (*) ont une valeur par défaut. Elles sont validées au démarrage dans src/config/env.ts.

```ts
const envSchema = z.object({
  NODE_ENV: z.enum(['development', 'production', 'test']).default('development'),
  PORT: z
    .string()
    .regex(/^\d+$/)
    .default('3000')
    .transform(Number),
  MONGODB_URI: z.string().url('MONGODB_URI must be a valid URL'),
  JWT_SECRET: z.string().min(32, 'JWT_SECRET must be at least 32 characters'),
  JWT_EXPIRES_IN: z.string().default('7d'),
  CORS_ORIGIN: z.string().default('*'),
});
```

Question 4

src/main.ts (point d'entrée), src/app.ts (configuration Express), src/server.ts (connexion à MongoDB et démarrage), le dossier config (variables, base, logs), les modèles User, Column et OF, les routes et contrôleurs, et les middlewares d'authentification, de validation et d'erreurs.



Question 5

Trois fichiers de tests existaient (santé du serveur, authentification, OF), soit 35 tests, mais ils ne pouvaient pas s'exécuter. Le fichier columns.test.ts était vide.

Question 6

Le projet est une API REST en couches (routes, contrôleurs, modèles, middlewares). Les principaux risques sont : tests et build cassés, aucune protection contre le brute force, absence d'en-têtes de sécurité, CORS ouvert à tous, URI MongoDB écrite dans les logs, 18 vulnérabilités dans les dépendances et aucun fichier de déploiement.

Priorités : réparer les tests et le build, compléter les tests, ajouter les protections de sécurité, conteneuriser, mettre en place la CI/CD et documenter. Le fichier tient sur une page.

| A1 | npm test échoue : pas d'env de test → process.exit(1) | Pas de CI possible |

| S1 | helmet / rate limit non branchés | Brute force sur /login |

| S4 | npm audit : 18 vulnérabilités (2 critiques) | Failles connues |

Étape 2 – Tests d'intégration

Question 7

Le fichier a été complété dans le dossier tests. Il utilise une base MongoDB en mémoire, vidée après chaque test, pour que les tests soient isolés.

Question 8

Pour GET : liste vide, colonnes renvoyées triées par position, refus sans token et avec un token invalide. Pour POST : création réussie, nouvelle colonne placée en dernière position, refus si le nom est absent ou déjà utilisé, refus sans token. Soit 9 tests.



```
it('renvoie les colonnes triées par position', async () => {
  await createTestColumn('Terminé', 2);
  await createTestColumn('À faire', 0);
  await createTestColumn('En cours', 1);

  const response = await request(app)
    .get('/api/columns')
    .set('Authorization', `Bearer ${authToken}`)
    .expect(200);

  expect(Array.isArray(response.body.data)).toBe(true);
  expect(response.body.data).toHaveLength(3);
  expect(response.body.data.map((c: { name: string }) => c.name)).toEqual([
    'À faire',
    'En cours',
    'Terminé',
  ]);
```

Question 9

Les codes attendus sont vérifiés (200, 201, 400, 401). La réponse doit contenir un message et un objet data avec l'identifiant, le nom, la position et les dates. Enfin, on interroge directement la base : la colonne existe après une création réussie, et rien n'est inséré en cas d'erreur.



```
const response = await request(app)
    .get('/api/columns')
    .set('Authorization', `Bearer ${authToken}`)
    .expect('Content-Type', /json/)
    .expect(200);

  expect(response.body).toEqual({
    message: 'Columns retrieved successfully',
    data: [],
  });
```

Question 10

Les variables de test ont été ajoutées dans vitest.config.ts et la commande npm test lance désormais une exécution unique. Résultat : 5 fichiers, **50 tests sur 50 réussis**.

vitest.config.ts + résultat

```
env: { NODE_ENV: 'test', JWT_SECRET: 'test-secret-key-at-least-32-characters', ... }

✓ tests/columns.test.ts (9 tests)

✓ tests/security.test.ts (6 tests)

Test Files  5 passed (5)

    Tests  50 passed (50)
```

Étape 3 – Conteneurisation

Question 11

Le Dockerfile se fait en deux étapes. La première installe toutes les dépendances et compile le TypeScript. La seconde, qui constitue l'image finale, n'installe que les dépendances de production, copie le code compilé et lance le serveur avec npm start. L'image utilise Node 20, tourne avec un utilisateur non administrateur et vérifie régulièrement la route /health.



```dockerfile
FROM node:20-alpine AS build
WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci

COPY tsconfig.json tsconfig.build.json ./
COPY src ./src
RUN npm run build


FROM node:20-alpine AS production
ENV NODE_ENV=production \
    PORT=3000
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev && npm cache clean --force
COPY --from=build /app/dist ./dist
USER node
EXPOSE 3000
CMD ["npm", "start"]

```

Question 12

La version de production a été lancée et testée : la route /health répond, une route protégée renvoie bien 401, et on peut créer un compte puis une colonne. Il reste à lancer docker compose up -d --build sur le poste pour valider la construction de l'image elle-même.

Commandes

```
docker compose up -d --build

curl http://localhost:3000/health      → {"status":"ok", ...}

curl -i http://localhost:3000/api/columns → 401 {"error":"No token provided"}
```

Question 13

Le fichier démarre deux services : MongoDB 7, avec un volume pour conserver les données, et l'API, qui attend que la base soit prête avant de démarrer. MongoDB n'est pas accessible depuis l'extérieur.



```yaml
services:
  mongo:
    image: mongo:7.0
    volumes: [mongo-data:/data/db]
  api:
    build: .
    env_file: .env
    environment:
      MONGODB_URI: mongodb://mongo:27017/visiplus
    ports: ["3000:3000"]
    depends_on:
      mongo: { condition: service_healthy }
```

Étape 4 – Stratégie CI/CD

Question 14

Automatiser l'installation, les tests et le build à chaque modification ; ne déployer qu'une version dont les tests passent ; contrôler la qualité (typage, tests, audit des dépendances) ; pouvoir revenir facilement à une version précédente.

Question 15

Un push lance les tests et le build. Une pull request vers `main` fait de même avec un audit de sécurité, et bloque la fusion en cas d'échec. Un push sur `main` déploie en recette, et une release déploie en production après validation. Les étapes sont : installation, vérification du typage et audit, tests, build de l'image Docker, publication puis déploiement.

Question 16

Un extrait GitHub Actions de 14 lignes figure dans `CI_CD_STRATEGIE.md` : il démarre un service MongoDB, installe Node 20, lance le typage, l'audit, les tests, le build et construit l'image Docker.



```yaml
on:
  push:
    branches: [main, 'feature/**']
  pull_request:
    branches: [main]
  release:
    types: [published]

jobs:
  ci:
    runs-on: ubuntu-latest
    services:
      mongo:
        image: mongo:7.0
        ports: ['27017:27017']
    env:
      MONGO_TEST_URI: mongodb://localhost:27017/visiplus_test
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - run: npm ci
      - run: npm run typecheck
      - run: npm audit --omit=dev --audit-level=high
      - run: npm test
      - run: npm run build
      - run: docker build -t $IMAGE:${{ github.sha }} .
```

Question 17

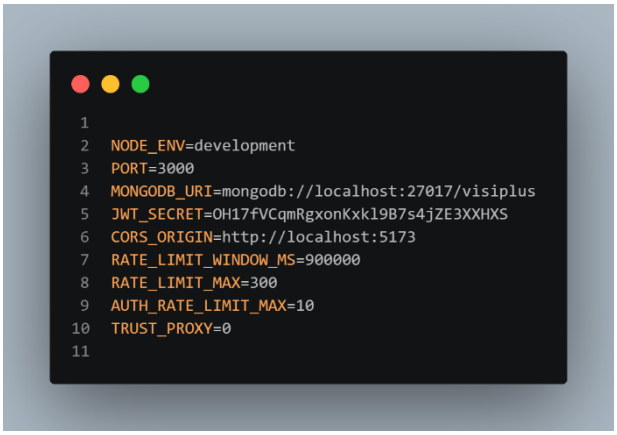
Aucun secret n'est dans le dépôt : le fichier .env est ignoré par Git et les vraies valeurs sont stockées dans les secrets GitHub. Avant la production, le pipeline doit être vert, la version testée en recette et le déploiement approuvé manuellement. Après déploiement, on vérifie /health, sinon on revient à la version précédente.



Étape 5 – Documentation du déploiement

Question 18

Node.js 20, npm, Docker avec Docker Compose, Git et MongoDB 7 (sauf si Docker est utilisé). Il faut créer un fichier .env à partir de .env.example en renseignant au minimum MONGODB_URI et un JWT_SECRET aléatoire.



Question 19

En local : npm ci, création du .env, puis npm run dev. Pour la production : npm run build puis npm start. Avec Docker : docker compose up -d --build démarre l'API et la base en une seule commande.

```
npm ci

copy .env.example .env

npm run dev

npm run build && npm start

docker compose up -d --build
```

Question 20

`npm test` lance tous les tests, `npm run test:watch` les relance à chaque modification et `npm run test:coverage` donne la couverture de code. Aucun MongoDB n'est nécessaire, la base est créée en mémoire.

```
npm test

npm run test:watch

npm run test:coverage
```

Question 21

Récupérer la nouvelle version avec `git pull`, relancer les tests, puis `docker compose up -d --build`. Les données sont conservées. Il est conseillé de sauvegarder la base avant, et de revenir au tag précédent en cas de problème.

```
git pull

npm ci && npm test

docker compose up -d --build

curl -f http://localhost:3000/health
```

Étape 6 – Sécurisation et bonnes pratiques

Question 22

Abus d'API : sans limite de requêtes, un attaquant peut essayer des milliers de mots de passe sur la route de connexion ou saturer le serveur. **Accès non autorisé et fuite d'informations** : sans en-têtes de sécurité, l'API est exposée au clickjacking, elle révèle qu'elle utilise Express et l'adresse de la base apparaissait dans les logs.

Question 23

Les deux ont été ajoutés dans `src/middlewares/security.middleware.ts`. Helmet ajoute les en-têtes de protection et masque la technologie utilisée. Express-rate-limit limite chaque adresse IP à 300 requêtes par 15 minutes sur l'API, et à 10 tentatives sur la connexion ; au-delà, l'API répond 429.



```ts
export function securityHeaders(): RequestHandler {
  return helmet({
    contentSecurityPolicy: {
      directives: {
        defaultSrc: ["'none'"],
        frameAncestors: ["'none'"],
      },
    },
    crossOriginResourcePolicy: { policy: 'same-site' },
  });
}

/**
 * Limiteur global
 */
export function apiRateLimiter(options: SecurityOptions = {}): RequestHandler {
  return rateLimit({
    windowMs: options.windowMs ?? env.RATE_LIMIT_WINDOW_MS,
    limit: options.apiMax ?? env.RATE_LIMIT_MAX,
    standardHeaders: 'draft-7',
    legacyHeaders: false,
    message: { error: 'Too many requests, please try again later.' },
  });

```

Question 24

Les middlewares sont branchés dans src/app.ts avant les routes. La taille des requêtes est aussi limitée à 100 ko, et les champs non prévus sont supprimés avant d'atteindre la base.



```ts
// --- Sécurité ---
app.disable('x-powered-by');
app.use(securityHeaders());
app.use('/api', apiRateLimiter(security));
app.use('/api/auth', authRateLimiter(security));

// Middleware
app.use(cors({ origin: env.CORS_ORIGIN }));
app.use(express.json({ limit: '100kb' }));
app.use(express.urlencoded({ extended: false, limit: '100kb' }));

```

Question 25

C'est vérifié par des tests automatiques : les routes des colonnes et des OF renvoient 401 sans token. Un autre test confirme qu'après trop de tentatives de connexion, l'API renvoie 429.



```
const response = await request(limitedApp)
  .post('/api/auth/login')
  .send(credentials)
  .expect(429);

expect(response.body).toHaveProperty('error');
expect(response.headers['ratelimit']).toBeDefined();
```

Question 26

La limitation est stockée en mémoire, ce qui convient à un seul serveur. Il n'y a pas encore de rôles (tout utilisateur connecté peut tout modifier), le CORS doit être restreint en production et le HTTPS doit être assuré par un reverse proxy. Les dépendances ont été mises à jour : 0 vulnérabilité.

Étape 7 – Synthèse et livraison

Question 27

Le dépôt contient le code, les tests, le Dockerfile, le docker-compose, le pipeline CI et les quatre documents Markdown. Le .gitignore exclut node_modules, dist et .env. Il suffit de faire git init, git add ., un commit et un tag v1.1.0.

Commandes

```
git init

git add .

git commit -m "feat: préparation mise en production"

git tag v1.1.0
```

Question 28

Le README présente l'application Kanban de suivi de production de Jaydee, les commandes pour l'installer et la lancer, la liste des routes, les choix techniques (tests, Docker, CI/CD), les mesures de sécurité, l'auteur (Thomas Tissier) et la version 1.1.0.